浙江大学实验报告

姓名：李彦萱　　专业：电子科学与技术　　学号：3250100761

课程名称：数字系统　　任课老师：李宇波

实验名称：(Prelab)Design On/Off Button　（Lab）Design an Electronic Lock　　实验日期：2026.3.24

1 实验目的和要求

1.1 实验目的

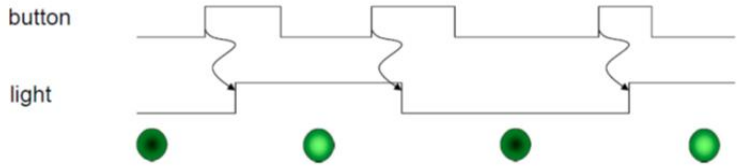
（1）Pre lab4:设计和实现 On/off button

（2）学习如何利用有限状态机（FSM）实现电子锁的控制

1.2 实验要求

Pre lab4:

根据课堂所学的开关按钮功能，实现以下功能



1）测试以下代码能否实现

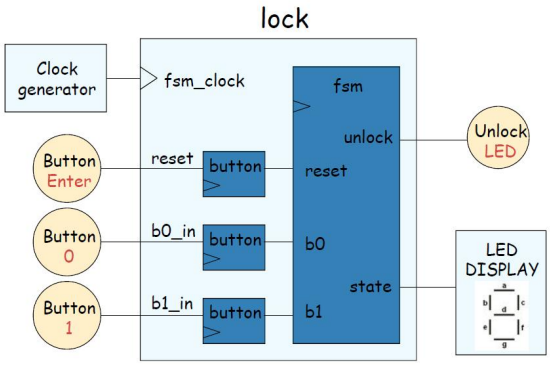
```
module onoff( input button ,output reg light );
    always @(posedge button)  light<= ~light ;
endmodule
```

2)编写正确的 Verilog 代码，并在板子上实现。（考虑亚稳态问题、抖动问题、脉宽变换）

3)模块中有三个处理子电路。试着去掉其中一部分，测试结果是否正确。

Lab4:

利用学过的知识设计电子锁如下：



1）画出 fsm 的状态图。

2) 编写 FSM 的行为级 Verilog 模块

3) 使用实验 3 中的方法编写一个仿真文件来模拟 FSM。

4) 写出上图中整个设计的 Verilog 描述，并在你的 Basys3 板上验证。

5)仅用 D 触发器和 and/OR/NOT 门绘制 FSM 电路。验证你编写的电路和 Vivado 合成的电路。

6)编写 FSM 的 Verilog 模块:对于下一状态逻辑和输出逻辑，用“assign”语句描述组合逻辑，对于 D 触发器，用“always”语句描述。

7)将步骤 2)中设计的 FSM 模块替换为步骤 2 中设计的 FSM 模块 6)，并在你的电路板上实现整个设计。

2 实验原理

Prelab4:

(1)完整功能需要三级处理电路：

消抖电路：

原理：通过计数器检测按键电平稳定时间，过滤掉抖动毛刺，输出稳定的按键电平信号。

实现：用系统时钟驱动计数器，当按键电平持续稳定超过阈值时，才更新输出状态。

(2)边沿检测电路：

原理：从稳定后的按键电平中提取上升/下降沿脉冲，作为一次有效的按键触发信号。

实现：用两级寄存器打拍，通过组合逻辑检测电平跳变，生成一个周期宽的脉冲信号。

(3)状态翻转电路：

原理：在有效按键脉冲触发下，翻转 LED 灯的亮灭状态。

实现：用 D 触发器，在有效脉冲的上升沿将 light 取反。

Lab4:

电子锁本质是一个有限状态机，根据输入密码序列和当前状态，决定下一个状态和输出（解锁/锁定）

状态编码：将每个状态映射为二进制编码（如 S0=000, S1=001...）。

用 D 触发器存储当前状态，用 AND/OR/NOT 门实现状态转移的组合逻辑，输出逻辑同样由组合门实现。

与 Vivado 综合后的电路对比，验证逻辑等价性。

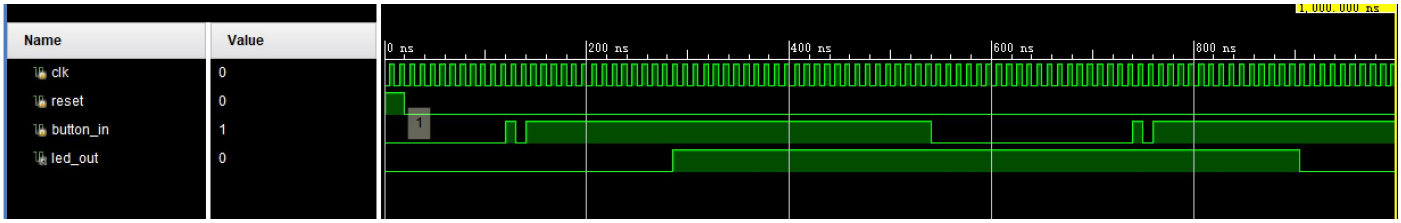
3 实验内容

Pre lab 4:

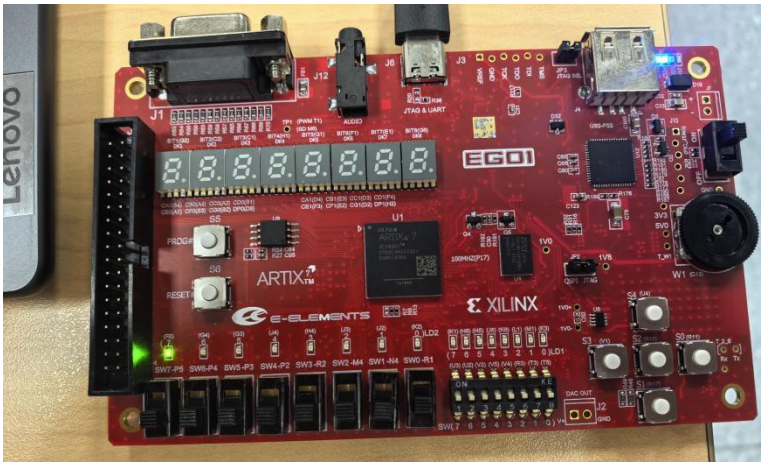
(1) 测试代码结果：没有去抖动的代码上板测试，发现按键输入后，灯的状态不稳定。（原因：由于机械触点的弹性作用，开关在闭合及断开的瞬间均伴随有一连串的抖动，导致灯的状态不稳定）。

(2) 实现按钮稳定输入的代码编写（代码赋在实验报告最后）

仿真如图所示：



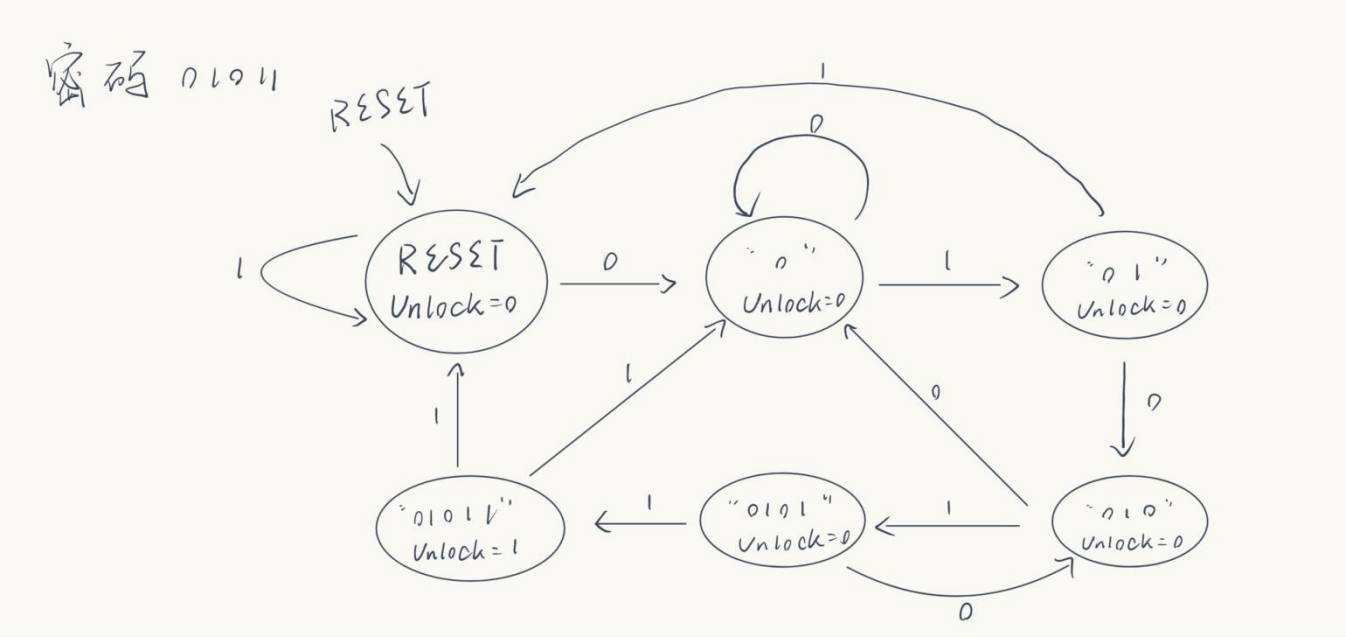
按钮按下后，LED 灯成功亮起，如图：



(3) 去掉其中一部分模块，发现仿真无法正常稳定运行

lab 4:

(1) 状态机图



分析：密码为 01011，状态图如上，在输入上我需要用 3 个按钮分别代表 reset,0,1；在输出上我要用一个 LED 亮起代表成功解锁即 unlock=1，用一个数码管的不同状态代表密码输入，即“reset”后数码管显示 0，“0”数码管显示 0，“01”数码管显示 01，以此推类“01011”为显示 5.其中如果下一位输入错误，则根据状态图退回，如在 0101 后输入 0 则退回“3”，而“0”后输入零退回“1”，“01”后输入“1”退回 0，其余类似。、

2)编写 FSM 的行为级 Verilog 模块

代码如下：

```
A.Module top_lock(
    input wire clk,
    input wire reset_btn,
    input wire btn_0,
    input wire btn_1,
    output wire unlock_led,
    output wire [6:0] seg,
    output wire [3:0] an
);
    wire b0_c, b0_p, b1_c, b1_p;

    debounce #(.NDELAY(650000)) d0 (.clk(clk), .reset(reset_btn), .noisy(btn_0), .clean(b0_c));
    pulse p0 (.clk(clk), .in(b0_c), .out(b0_p));

    debounce #(.NDELAY(650000)) d1 (.clk(clk), .reset(reset_btn), .noisy(btn_1), .clean(b1_c));
    pulse p1 (.clk(clk), .in(b1_c), .out(b1_p));

    wire [3:0] step;
    lock_fsm                                                                    my_fsm
(.clk(clk),.reset(reset_btn),.btn0_pulse(b0_p),.btn1_pulse(b1_p),.unlock(unlock_led),.state_num(s
tep));
    seg7_decoder my_seg (.num(step), .seg(seg), .seg_en(an));
endmodule

B.module lock_fsm(
    input clk,
```

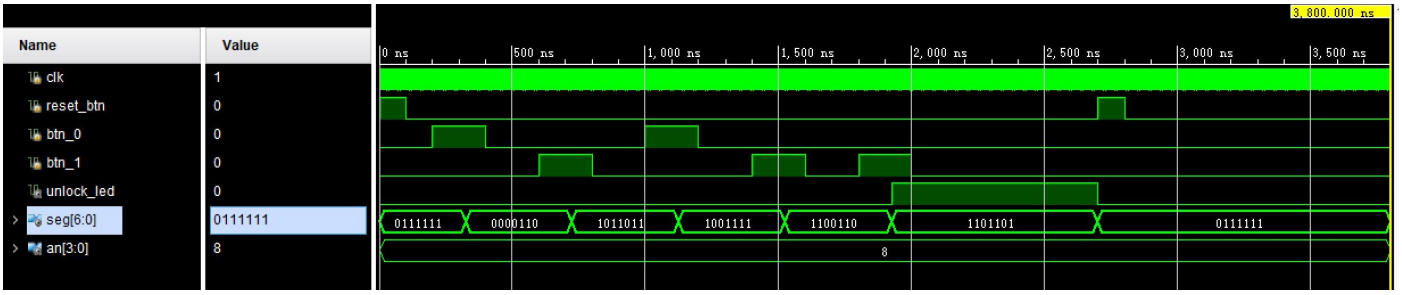
```
    input reset,
    input btn0_pulse,
    input btn1_pulse,
    output reg unlock,
    output reg [3:0] state_num
);
    parameter S_RESET = 3'd0, S_1 = 3'd1, S_2 = 3'd2, S_3 = 3'd3, S_4 = 3'd4, S_5 = 3'd5;
    reg [2:0] curr, next;

    always @(posedge clk or posedge reset) begin
        if (reset) curr <= S_RESET;
        else curr <= next;
    end

    always @(*) begin
        next = curr;
        if (btn0_pulse) begin
            case(curr)
                S_RESET: next = S_1;
                S_1:     next = S_1;
                S_2:     next = S_3;
                S_3:     next = S_1;
                S_4:     next = S_3;
                S_5:     next = S_1;
            endcase
        end else if (btn1_pulse) begin
            case(curr)
                S_RESET: next = S_RESET;
                S_1:     next = S_2;
                S_2:     next = S_RESET;
                S_3:     next = S_4;
                S_4:     next = S_5;
                S_5:     next = S_RESET;
            endcase
        end
    end

    always @(*) begin
        unlock = (curr == S_5);
        state_num = {1'b0, curr};
    end
endmodule

C.module debounce #(
    parameter NDELAY =10,
    parameter NBITS = 20
)(
    input wire clk, input wire reset, input wire noisy,
    output reg clean
```

```
);
    reg [NBITS-1:0] count;
    reg xnew;
    always @(posedge clk) begin
        if (reset) begin xnew <= noisy; clean <= noisy; count <= 0; end
        else if (noisy != xnew) begin xnew <= noisy; count <= 0; end
        else if (count == NDELAY) begin clean <= xnew; end
        else begin count <= count + 1; end
    end
Endmodule

D.module pulse(
    input wire clk,
    input wire in,
    output wire out
);
    reg temp = 0;
    always @(posedge clk) begin
        temp <= in;
    end
    assign out = (~temp) & in;

Endmodule

module seg7_decoder(
    input  wire [3:0] num,
    output reg  [6:0] seg,
    output wire [3:0] seg_en
);

    assign seg_en = 4'b1000;

    always @(*) begin
        case(num)

            4'd0: seg = 7'b0111111;
            4'd1: seg = 7'b0000110;
            4'd2: seg = 7'b1011011;
            4'd3: seg = 7'b1001111;
            4'd4: seg = 7'b1100110;
            4'd5: seg = 7'b1101101;
            default: seg = 7'b0000000;
        endcase
    end
Endmodule
```

3)仿真实验

A. 密码正确时

仿真输出如图：灯被成功点亮。



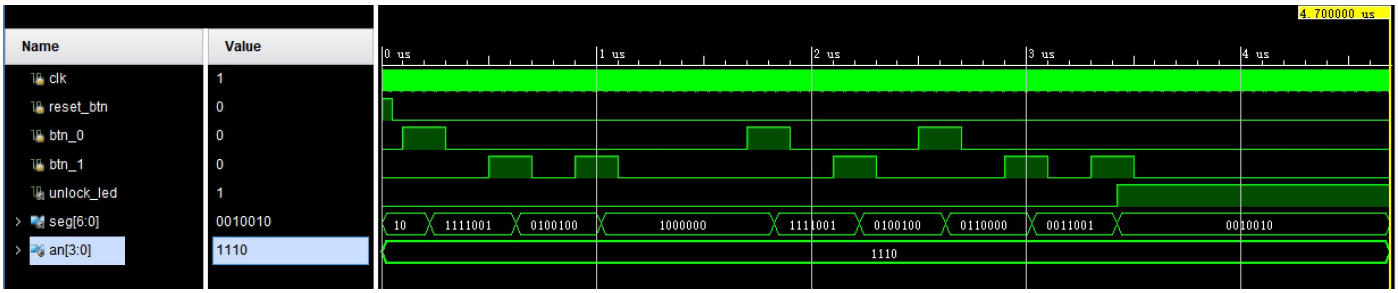
代码如下：

```
module top_lock_tb();

    reg clk;
    reg reset_btn; //  对应  S3
    reg btn_0;        //  对应  S2
    reg btn_1;        //  对应  S0

    wire unlock_led;
    wire [6:0] seg;
    wire [3:0] an;


    top_lock uut (
        .clk(clk),
        .reset_btn(reset_btn),
        .btn_0(btn_0),
        .btn_1(btn_1),
        .unlock_led(unlock_led),
        .seg(seg),
        .an(an)
    );

    defparam uut.d0.NDELAY = 10;
    defparam uut.d1.NDELAY = 10;

    always #5 clk = ~clk;

    task press_0;
        begin
            btn_0 = 1; #200; //  按下  200ns
            btn_0 = 0; #200; //  松开  200ns
        end
    endtask

    task press_1;
        begin
            btn_1 = 1; #200;
            btn_1 = 0; #200;
        end
    endtask
```

```
    initial begin
        clk = 0;
        reset_btn = 1;
        btn_0 = 0;
        btn_1 = 0;

        #100;
        reset_btn = 0;
        #100;
        press_0(); // 第一位：0
        press_1(); // 第二位：1
        press_0(); // 第三位：0
        press_1(); // 第四位：1
        press_1(); // 第五位：1
        #500;

        reset_btn = 1;
        #100;
        reset_btn = 0;

        #1000;
        $stop;
    end
endmodule
```

B. 错误密码输入后正确（01101001）

输出波形图如下：在起初错误输入后

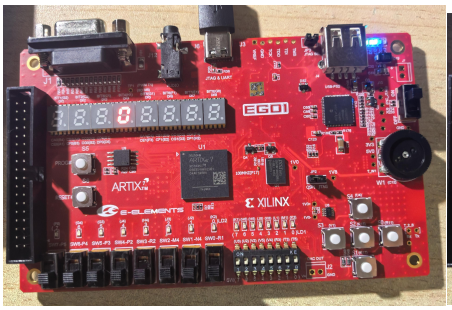
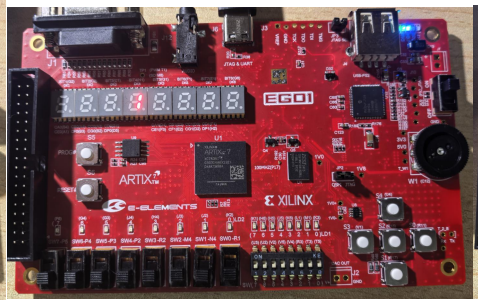
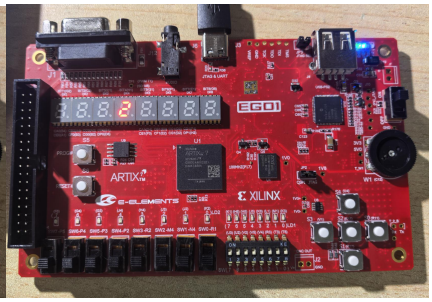
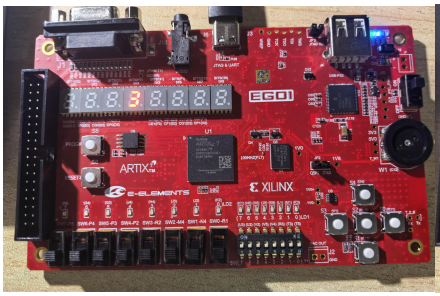
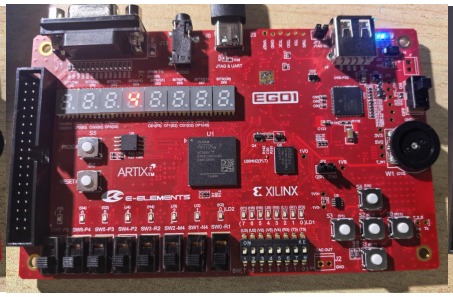
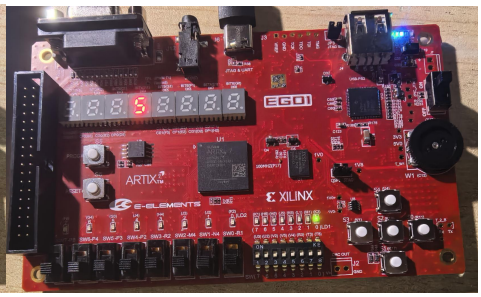


代码如下：

```
module top_lock_tb();
    reg clk;
    reg reset_btn;
    reg btn_0;
    reg btn_1;

    wire unlock_led;
    wire [6:0] seg;
    wire [3:0] an;

    top_lock uut (
        .clk(clk),
        .reset_btn(reset_btn),
        .btn_0(btn_0),
```

```
        .btn_1(btn_1),
        .unlock_led(unlock_led),
        .seg(seg),
        .an(an)
    );

    defparam uut.d0.NDELAY = 10;
    defparam uut.d1.NDELAY = 10;

    always #5 clk = ~clk;
    task press_0;
        begin
            btn_0 = 1; #200;
            btn_0 = 0; #200;
        end
    endtask

    task press_1;
        begin
            btn_1 = 1; #200;
            btn_1 = 0; #200;
        end
    endtask

    initial begin
        clk = 0; reset_btn = 1; btn_0 = 0; btn_1 = 0;
        #100; reset_btn = 0; #100;
        press_0();
        press_1();
        press_1();

        #600;
        press_0();
        press_1();
        press_0();
        press_1();
        press_1();

        #1000;
        $stop;
    end
Endmodule
```

4）上版实验（输入 01011）

5)手动计算 FSM，以仅使用 D 触发器和 and/OR/NOT 门绘制 FSM 的电路。

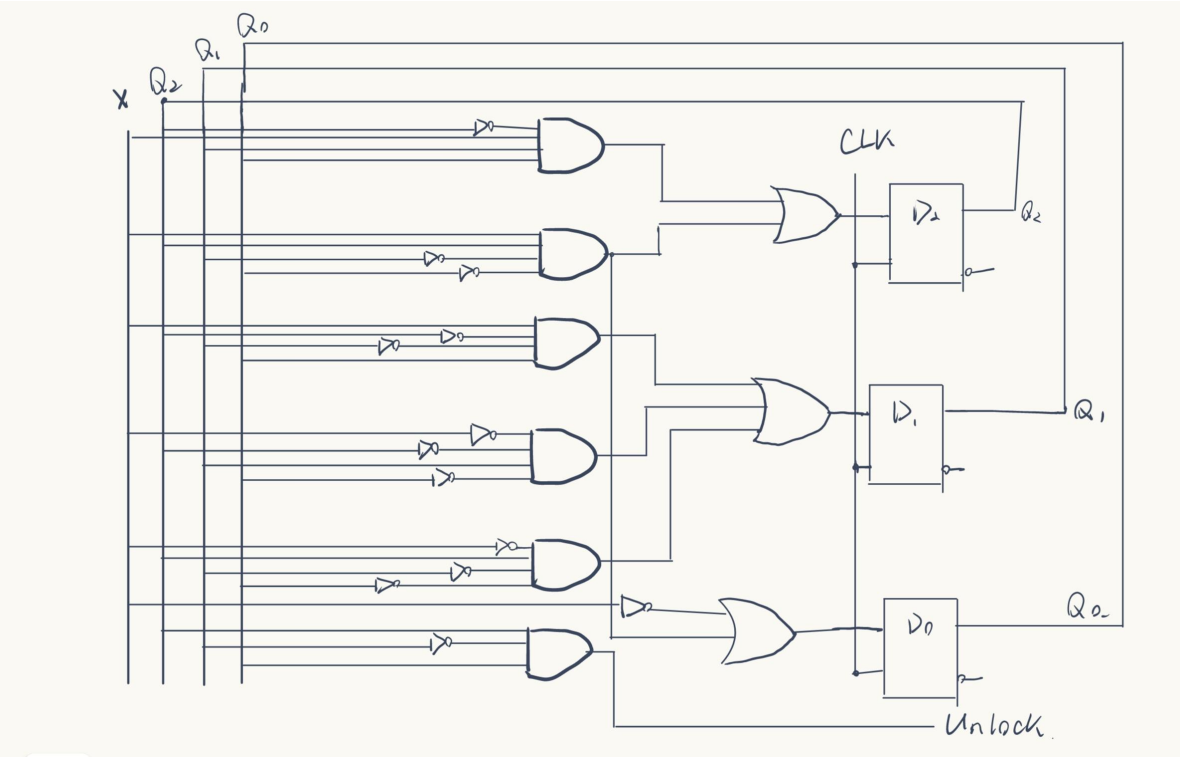
状态			输入 X	次态
S_RESET	000	000	0	001
		000	1	000
S_1	001	001	0	001
S_2	010	001	1	010
S_3	011	010	0	011
		010	1	000
S_4	100	011	0	001
S_5	101	011	1	100
		100	0	011
		100	1	101
		101	0	001
		101	1	000

$D_2 = Q_2' Q_1 Q_0 X + Q_2 Q_1' Q_0' X$

$D_1 = Q_2' Q_1' Q_0 X + Q_2' Q_1 Q_0' X' + Q_2 Q_1' Q_0' X'$

$D_0 = X' + Q_2 Q_1' Q_0' X$

$Unlock = Q_2 Q_1' Q_0$

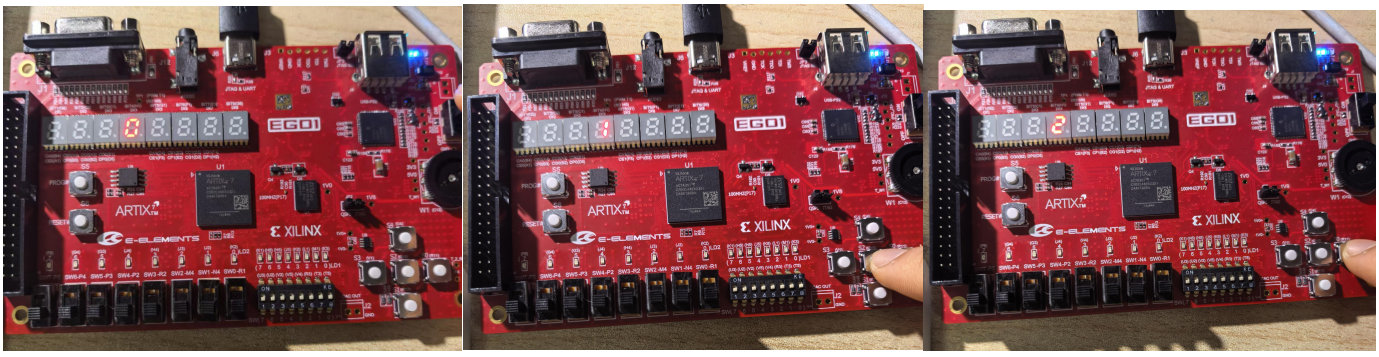


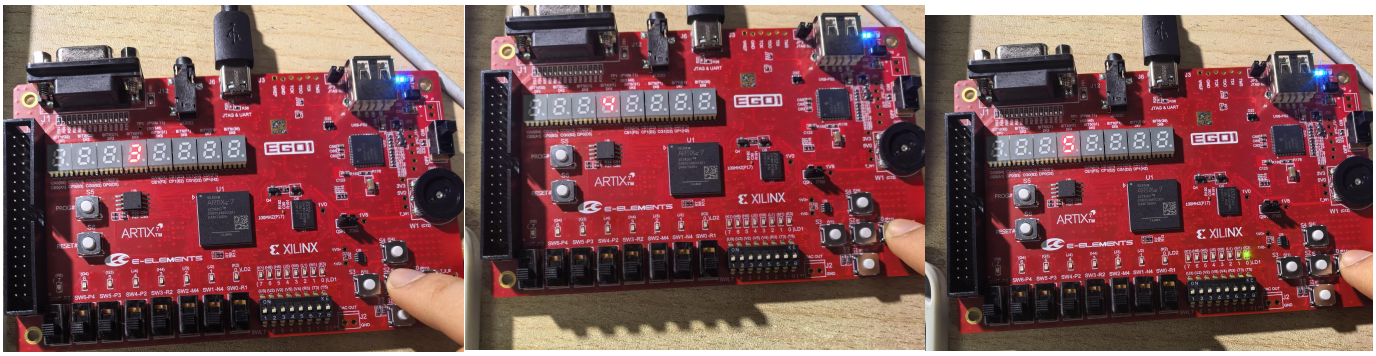
6）使用时序逻辑和组合逻辑代码实现

```
module lock_fsm(
    input wire clk,
    input wire reset,
    input wire btn0_pulse,
    input wire btn1_pulse,
    output wire unlock,
    output wire [3:0] state_num
);
    reg Q2, Q1, Q0;

    wire D2, D1, D0;
    wire hold = ~(btn0_pulse | btn1_pulse);
    wire N2_0 = 1'b0;
    wire N1_0 = (~Q2 & Q1 & ~Q0) | (Q2 & ~Q1 & ~Q0);
    wire N0_0 = 1'b1;
    wire N2_1 = (~Q2 & Q1 & Q0) | (Q2 & ~Q1 & ~Q0);
    wire N1_1 = (~Q2 & ~Q1 & Q0);
    wire N0_1 = (Q2 & ~Q1 & ~Q0);
    assign D2 = (hold & Q2) | (btn0_pulse & N2_0) | (btn1_pulse & N2_1);
    assign D1 = (hold & Q1) | (btn0_pulse & N1_0) | (btn1_pulse & N1_1);
    assign D0 = (hold & Q0) | (btn0_pulse & N0_0) | (btn1_pulse & N0_1);
    assign unlock = Q2 & ~Q1 & Q0;
    assign state_num = {1'b0, Q2, Q1, Q0};
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            Q2 <= 1'b0;
            Q1 <= 1'b0;
            Q0 <= 1'b0;
        end else begin
```

```
                Q2 <= D2;
                Q1 <= D1;
                Q0 <= D0;
            end
        end
Endmodule

module seg7_decoder(
        input    wire [3:0] num,
        output reg    [6:0] seg,
        output wire [3:0] seg_en
);

        assign seg_en = 4'b1000;
        always @(*) begin
            case(num)
                4'd0: seg = 7'b0111111;
                4'd1: seg = 7'b0000110;
                4'd2: seg = 7'b1011011;
                4'd3: seg = 7'b1001111;
                4'd4: seg = 7'b1100110;
                4'd5: seg = 7'b1101101;
                default: seg = 7'b0000000;
            endcase
        end
endmodule
```

7）上板实验





4 实验结果和分析

实验顺利完成，结果如上。

5. 心得与体会

本次实验，我更深入地理解了状态机的原理，理解原理、编写程序的能力也得到了提升。起初我在编写程序时经历多次报错，经检查是数码管的引没对齐。后来在第一次成功上板编译时发现所有该不亮的都亮了，该亮的反而都没亮，又经过修改才正常输出。修改代码是一个长期而细致的过程，需要耐性。

附件：原代码

Prelab4:

模块代码：

```
module top_onoff(
    input    wire clk,
    input    wire reset,
    input    wire button_in,
    output reg    led_out = 0
);
    // 防止亚稳态
    reg b1, b2;
    always @(posedge clk) begin
        b1 <= button_in;
        b2 <= b1;
    end
    //去抖
    wire b_clean;
    debounce u_debounce (
        .clk(clk),
        .reset(reset),
        .noisy(b2),
        .clean(b_clean)
    );
    //脉宽变换
    wire b_pulse;
    pulse u_pulse (
        .clk(clk),
        .in(b_clean),
        .out(b_pulse)
    );
    //开关翻转逻辑
    always @(posedge clk) begin
        if (reset) led_out <= 0;
        else if (b_pulse) led_out <= ~led_out;
    end
Endmodule

module debounce(
    input wire clk,
    input wire reset,
    input wire noisy,
    output reg    clean
);
```

```
    parameter NDELAY = 10;
    parameter NBITS = 20;

    reg [NBITS-1:0] count = 0;
    reg xnew = 0;

    always @(posedge clk) begin
        if (reset) begin
            xnew <= 0; clean <= 0; count <= 0;
        end
        else if (noisy != xnew) begin
            xnew <= noisy;
            count <= 0;
        end
        else if (count == NDELAY) begin
            clean <= xnew; //更新输出
        end
        else begin
            count <= count + 1;
        end
    end
Endmodule

module pulse(
    input wire clk,
    input wire in,
    output wire out
);
    reg temp = 0;
    always @(posedge clk) begin
        temp <= in;
    end
    assign out = in & (~temp);
Endmodule
```

仿真代码：

```
module top_onoff_tb();
    reg clk, reset, button_in;
    wire led_out;
    top_onoff #(.NDELAY(10)) uut (
        .clk(clk),
        .reset(reset),
        .button_in(button_in),
        .led_out(led_out)
    );

    always #5 clk = ~clk;
```

```
    initial begin

            clk = 0; reset = 1; button_in = 0;
            #20 reset = 0;

            //第一次按下
            #100;
            button_in = 1;
            #10; button_in = 0; #10; button_in = 1; //  模拟抖动
            #400;                       // 保持按下状态，观察  led_out  变  1
            button_in = 0;          //  松开按键

            #200;

            //  第二次按下
            button_in = 1;          //按下
            #10; button_in = 0; #10; button_in = 1; //模拟抖动
            #400;                       // 保持按下状态，预期观察  led_out  变  0
            button_in = 0;          //  再次松开

            #200;
            $display("Toggle On/Off Test Finished!");
            $stop;
        end
Endmodule
```

LAB4

约束文件：

```
# 时钟
set_property PACKAGE_PIN P17 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
# 按钮
set_property PACKAGE_PIN V1 [get_ports reset_btn]
set_property IOSTANDARD LVCMOS33 [get_ports reset_btn]
set_property PACKAGE_PIN R15 [get_ports btn_0]
set_property IOSTANDARD LVCMOS33 [get_ports btn_0]
set_property PACKAGE_PIN R11 [get_ports btn_1]
set_property IOSTANDARD LVCMOS33 [get_ports btn_1]
# 解锁指示灯  LD0
set_property PACKAGE_PIN K3 [get_ports unlock_led]
set_property IOSTANDARD LVCMOS33 [get_ports unlock_led]
# 数码管段选
set_property PACKAGE_PIN B4 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN A4 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN A3 [get_ports {seg[2]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN B1 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN A1 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN B3 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN B2 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
# 数码管位选
set_property PACKAGE_PIN G2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN C2 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN C1 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN H1 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```